FieldFlow Business Requirements Doc.

Kurshat Muhammet & Zhuyutong Li

Summary Problem Description:

Field service companies like HVAC, plumbing, electrical, and facilities maintenance continue to rely on outdated methods such as spreadsheets and phone calls to coordinate their technicians. The methods may have been sufficient in simpler times, but they are no longer equipped to handle the complexity of modern day operations. Minor errors in spreadsheets isn't the only problem. Scheduling conflicts arise, skill mismatches occur with lack of real-time insight into technician qualifications, service history becomes unreliable, and the system entirely lacks efficiency as availability is tracked manually.

Beyond internal inefficiencies, clients are left in the dark. With fragmented systems, they're unable to track the status of their service requests or provide structured feedback once a job is complete. Meanwhile, operations managers have no reliable way to measure technician utilization, identify bottlenecks, or make data-driven decisions about workload distribution.

FieldFlow addresses these challenges by providing a structured database system that manages the full lifecycle of a work order. The moment a client submits a service request, through strategized technician dispatch and job execution to final completion and client review, FieldFlow enforces the constraints that manual systems cannot. FieldFlow ensures that the right technician, with the right skills, reaches the right location at the right time.

The system was first designed as a relation database (Project 1) and then migrated to a MongoDB document store with two root collections: **users** and **jobs** (Project 2). In project 3, we will extend FieldFlow with a Redis in-memory key-value layer to support three high-performance, real-time features that benefit from quick read/write access.

Business Rules:

A **Client** *submits* one or more **Jobs** describing the work needed.
A **Dispatcher** *dispatches* exactly one **Job** at a time; a **Job** is *dispatched* by one **Dispatcher**.
A **Technician** *is assigned to* zero or more **Jobs**; a **Job** is *assigned to* at most one **Technician**.
A **Technician** *possesses* one or more **Skills**; a **Skill** can be *possessed* by many **Technicians**.
A **Job** *requires* one or more **Skills** through the **Technician-Skill** matching constraint. A **Job** may *have* zero or one **Review**; a **Review** belongs to exactly one **Job**.
A **Region** *contains* one or more **Technicians**; a **Technician** belongs to exactly one **Region**.
A **Region** *serves* one or more **Clients**; a **Client** belongs to exactly one **Region**.
A **Technician** tracks an availability status and an average rating derived from **Job** reviews. A **Job** tracks its status (e.g., assigned, in progress, completed), scheduled date, completed date, and priority level.
Client, **Technician**, and **Dispatcher** are all specializations of **Person**, sharing common attributes (name, email, phone)

Identified Actions (possible verbs/relationships):

Client *submits* **Job**
Dispatcher *dispatches* **Job**
Technician *is assigned to* **Job**
Technician *possesses* **Skill**

Job *requires* **Skill**
Job *has* **Review**
Region *contains* **Technician**
Region *serves* **Client**
Client, **Technician**, **Dispatcher** *is a* **Person**

Relational Schema

Primary Keys = underlined // Foreign Keys = *italicized*

Person(personId, name, email, phone)

Client(personId, address, *regionId*)

Dispatcher(personId, department)

Technician(personId, isAvailable, *regionId*)

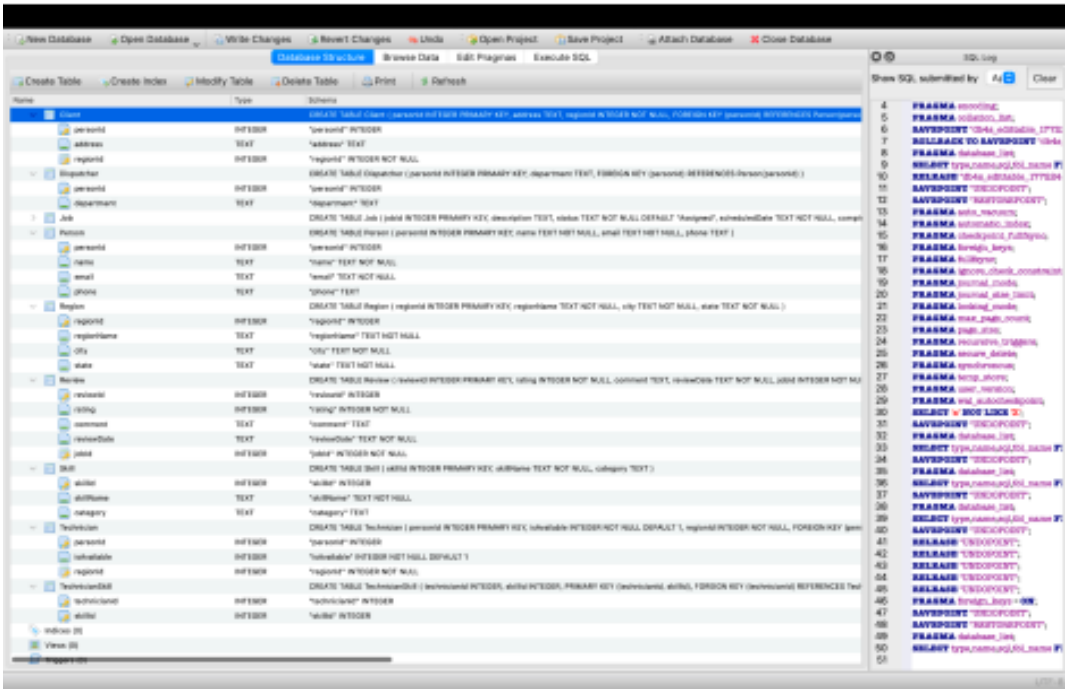
Region(regionId, regionName, city, state)

Skill(skillId, skillName, category)

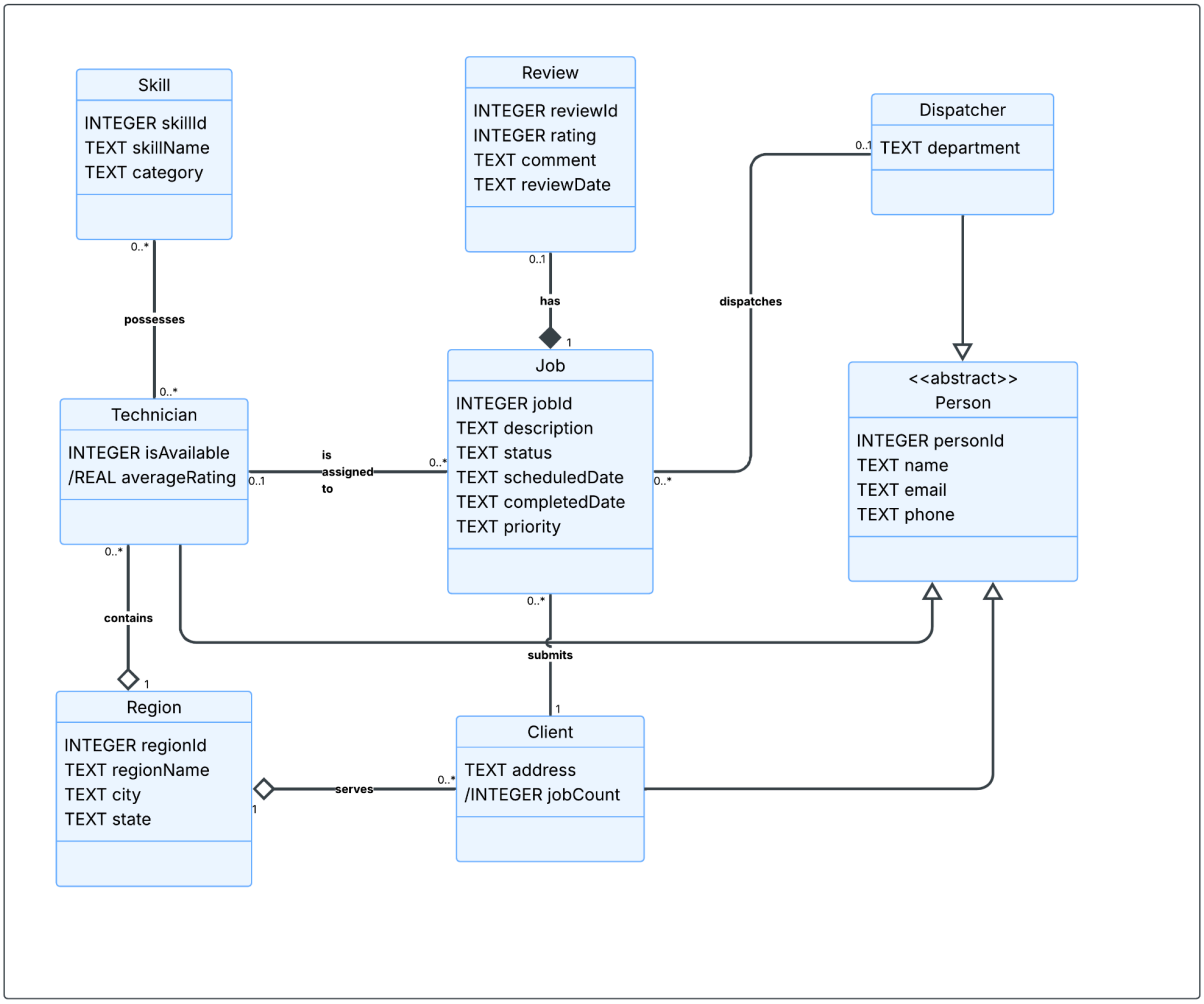
TechnicianSkill(technicianId, skillId)

Job(jobId, description, status, scheduledDate, completedDate, priority, *clientId*, *technicianId*, *dispatcherId*)

Review(reviewId, rating, comment, reviewDate, *jobId*)



FieldFlow UML Diagram



MongoDB Layer		**Redis Layer**
Users collection (technicians, clients, dispatchers)	→ feeds →	Hash: tech:{name} Sorted Set: leaderboard
Jobs collection (all work orders)	→ feeds →	List: jobQueue

Redis Additions

1. **Active Job Queue (List)**

Dispatchers need to see pending jobs in the order they arrived so technicians can be assigned quickly. A redis list would preserve the insertion order and support queue-style push/pop operations.

2. **Technician Leaderboard (Sorted Set)**

Management wants a live ranking of technicians by number of completed jobs. A Redist sorted set stores technician names as members with their completed job count as the score, enabling instant ranked queries without hitting MongoDB.

3. **Technician Profile Cache (Hash)**

Frequently accessed technician details (e.g. name, phone, region, availability) are cached as Redis hashes keyed by technician name. This reduces MongoDB reads for high-traffic lookups like dispatcher dashboards.